



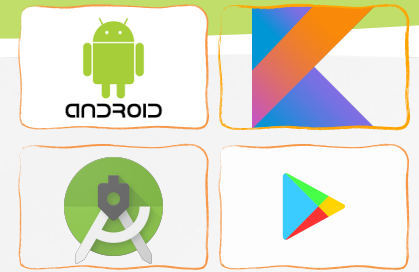
Software Development for Mobile Devices

Nguyen Le Hoang Dung
[*nlhdung@fit.hcmus.edu.vn*](mailto:nlhdung@fit.hcmus.edu.vn)



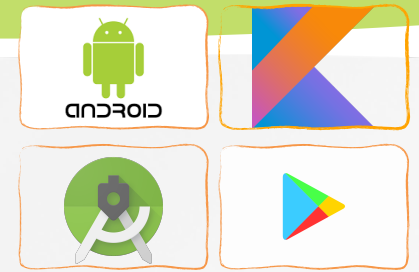
Content

- » Android app's fundamentals
- » Anatomy of Android applications
- » Component's life cycle
- » Activity life cycle
- » Localization



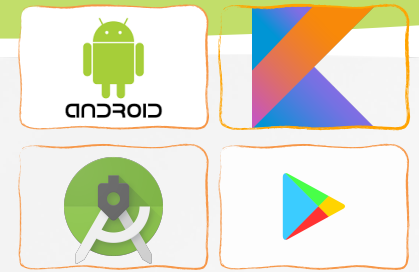
Android App's Fundamentals

- » They are usually created using the Java/**Kotlin**...
- » Apps can use various Android Libraries (such as android.jar, maps.jar,...) to gain the functionality needed to work inside the Android OS.
- » Android apps are made of multiple elements such as: user-defined classes, android jars, third-party libraries, XML files defining the UIs or views, multimedia resources, data assets such as strings, databases... and finally a Manifest summarizing the 'anatomy' and permissions requested by it.



Android App's Fundamentals (cont.)

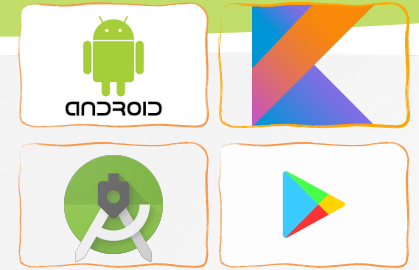
- » The various app components are given to the compiler to obtain a single signed and deployable Android Package (.apk file).
- » Like “.class” files in Java, “.apk” files are the byte-code version of the app that finally will be ‘executed’ by interpretation inside either a Dalvik Virtual Machine (DVM) or an Android-Runtime Engine (ART).



Android App's Fundamentals (cont.)

- » Each Android app lives in its own security sandbox, protected by the following Android security features:
 - » The Android operating system is a multi-user Linux system in which **each app** is a **different user**
 - » By default, the system assigns each app a unique Linux user ID. The system sets **permissions for all the files** in an app so that only the user ID assigned to that app can access them.
 - » Each process has its own virtual machine (VM), so an app's code runs in **isolation** from other apps.

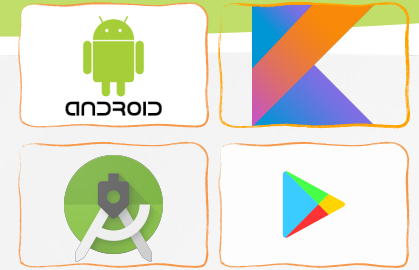
Android App's Fundamentals (cont.)



- » App to share data with other apps and for an app to access system services:
 - » Apps with the same user ID can also arrange to run in the same Linux process and share the same VM. The apps must also be signed with the same certificate.
 - » An app can request permission to access device data such as the device's location, camera, and Bluetooth connection. The user has to explicitly grant these permissions.

Android App's Anatomy

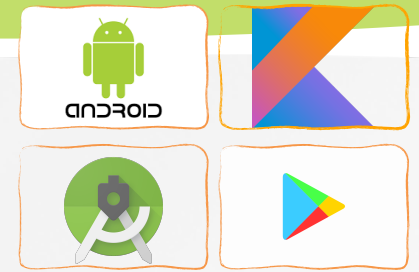
Core Components



- » Core components are the primordial classes or building blocks from which apps are made.
- » Each core component provides a particular type of functionality and has a distinct lifecycle. A lifecycle defines how the component is created, transitioned, and destroyed.
- » There are four type of core components
 - » An Activity
 - » A Service
 - » A broadcast receiver
 - » A content provider

Android App's Anatomy

Core Components (cont.)

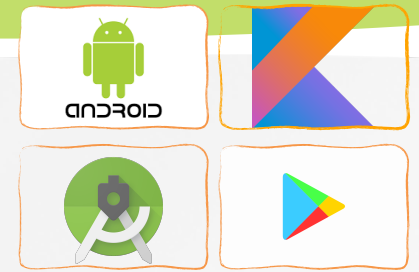


» Activity Class

- » An Activity object is similar to a WindowsForm. It usually presents a single graphical visual interface (GUI) which in addition to the displaying/collecting of data, provides some kind of 'code-behind' functionality.
- » A typical Android application contains one or more Activity objects.
- » Applications must designate one activity as their main task or entry point. That activity is the first to be executed when the app is launched.
- » An activity may transfer control and data to another activity through an interprocess communication protocol called **intents**.

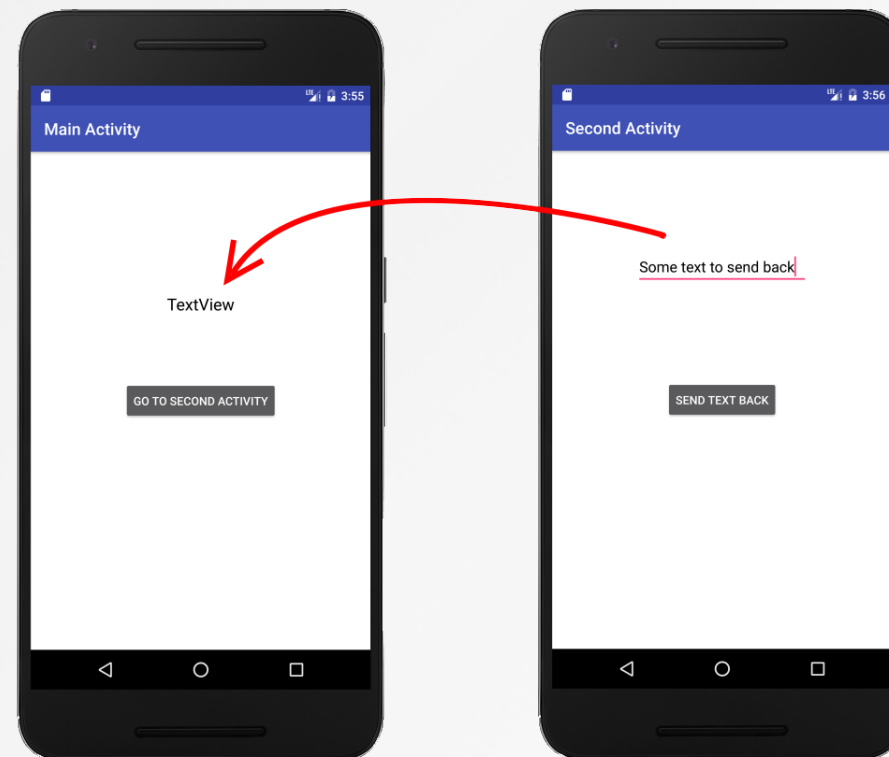
Android App's Anatomy

Core Components (cont.)



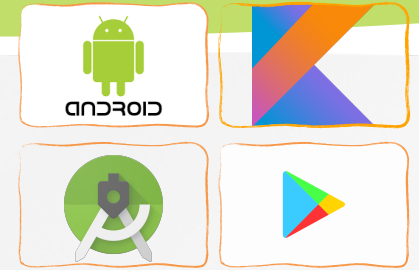
» Activity Class

» Example of an app containing 2 Activities



Android App's Anatomy

Core Components (cont.)

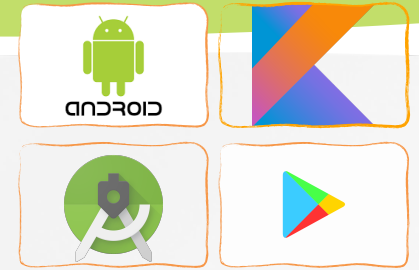


» Service Class

- » Services are a special type activity that do not have a visual user interface. A service object may be active without the user noticing its presence.
- » Services are analogous to secondary threads, usually running some kind of background 'busy-work' for an indefinite period of time.
- » Applications start their own services or connect to services already active.

Android App's Anatomy

Core Components (cont.)

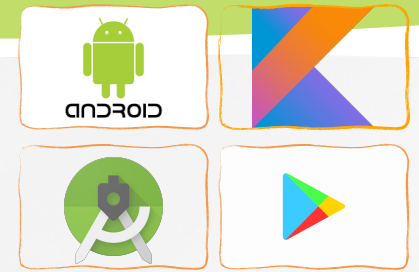


» Broadcast Receiver Class

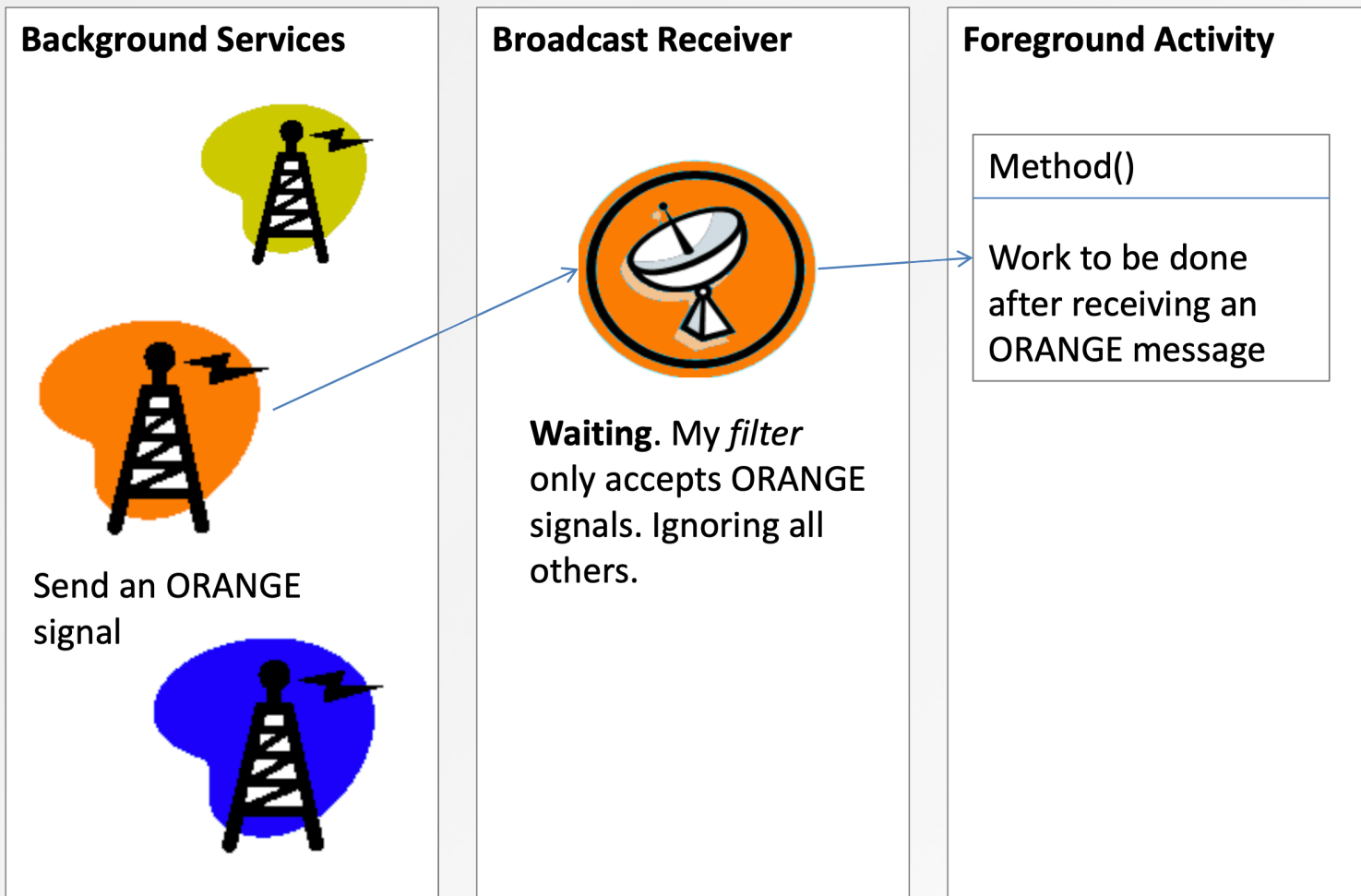
- » A BroadcastReceiver is a dedicated listener that waits for a triggering system-wide message to do some work. The message could be something like: low-battery, wi-fi connection available, earth-quakes in California, speed-camera nearby.
- » Broadcast receivers do not display a user interface.
- » They typically register with the system by means of a filter acting as a key. When the broadcasted message matches the key the receiver is activated.
- » A broadcast receiver could respond by either executing a specific activity or use the notification mechanism to request the user's attention.

Android App's Anatomy

Core Components (cont.)

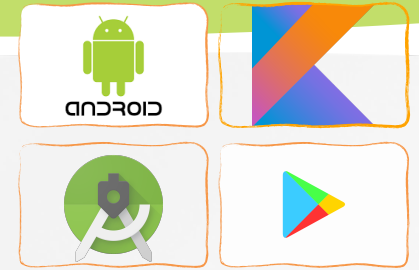


» Broadcast Receiver Class



Android App's Anatomy

Core Components (cont.)

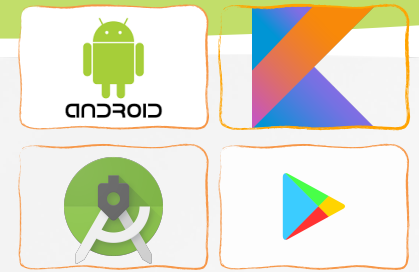


» Content Provider Class

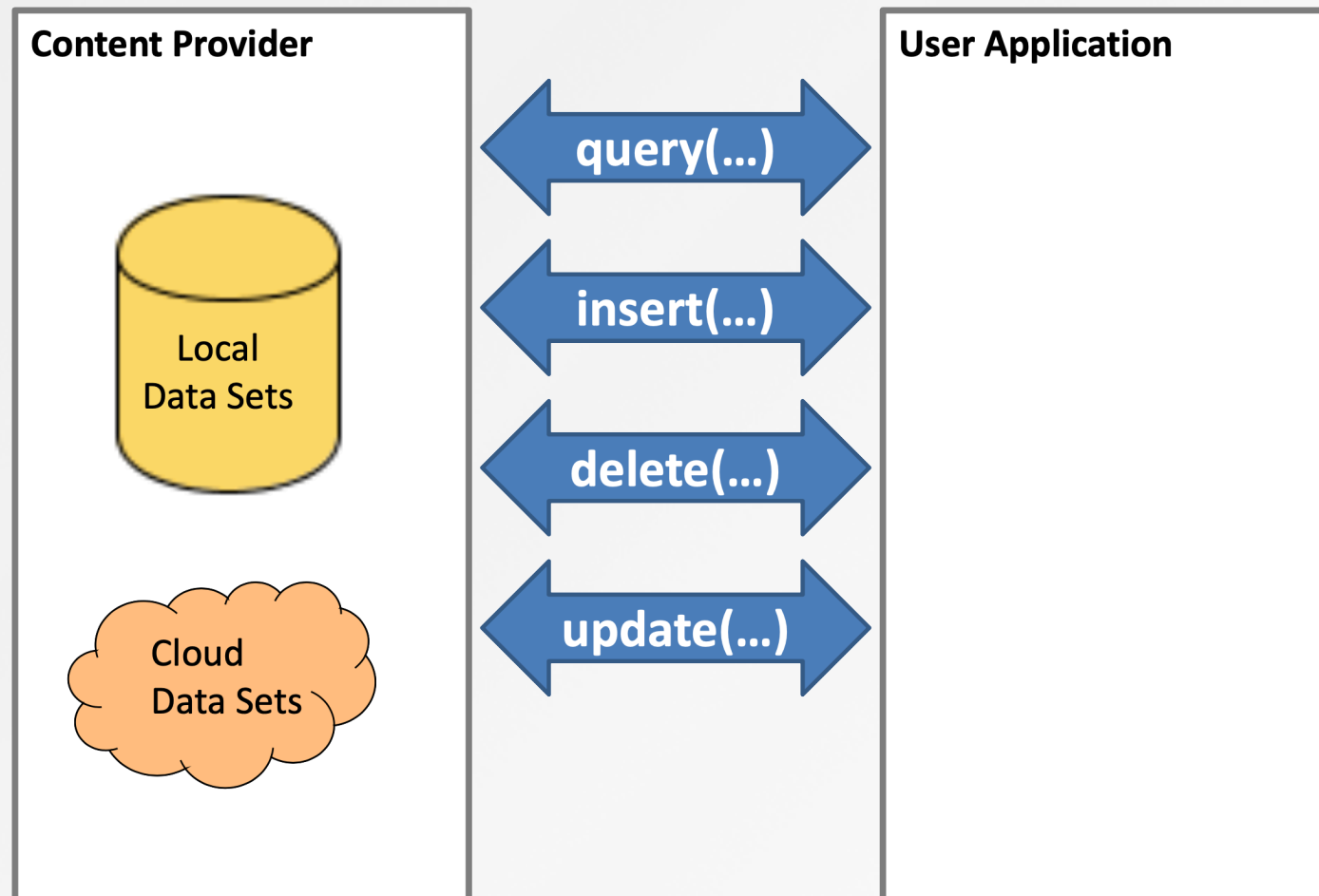
- » A content provider is a data-centric service that makes persistent datasets available to any number of applications.
- » Common global datasets include: contacts, pictures, messages, audio files, emails.
- » The global datasets are usually stored in a SQLite database (however the developer does not need to be an SQL expert)
- » The content provider class offers a standard set of parametric methods to enable other applications to retrieve, delete, update, and insert data items.

Android App's Anatomy

Core Components (cont.)

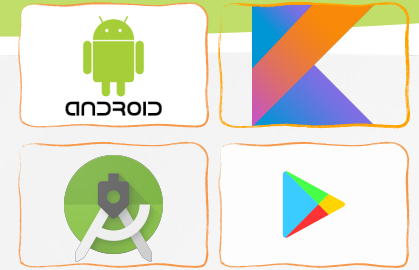


» Content Provider Class

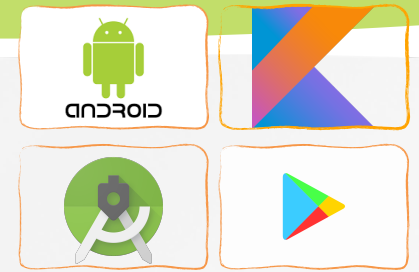


Android App's Anatomy

Core Components (cont.)



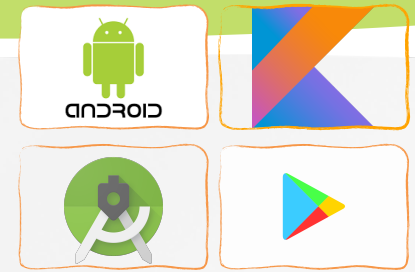
- » Activities, services, and broadcast receivers are activated by an asynchronous message called an *intent*.
 - » An intent is created with an Intent object.
- » There are separate methods for activating each type of component:
 - » An Intent to **startActivity()** or **startActivityForResult()**
 - » An Intent to **startService()** or **bindService()**
 - » An Intent to **sendBroadcast()**, **sendOrderedBroadcast()**, or **sendStickyBroadcast()**
 - » Perform a query to a content provider by calling **query()** on a **ContentResolver**



Component's Life Cycle

Life and Death in Android

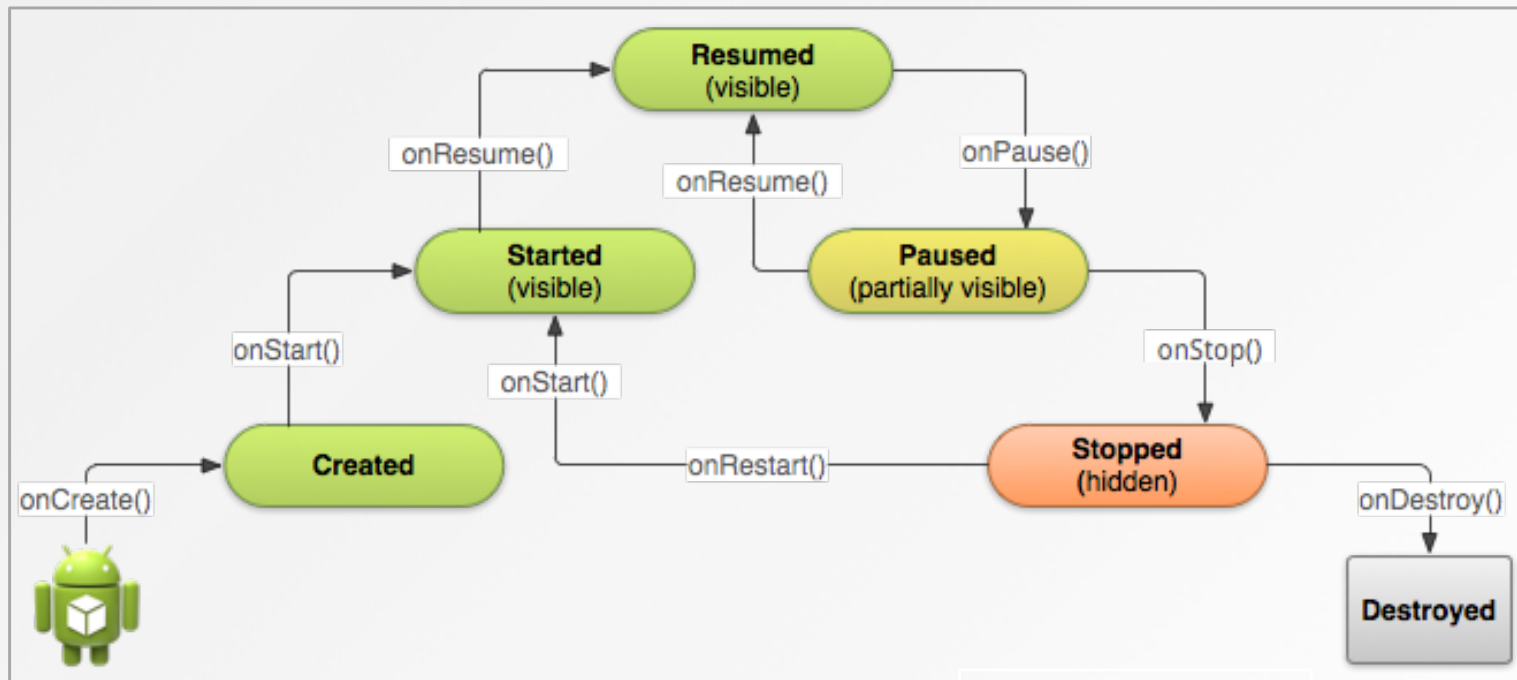
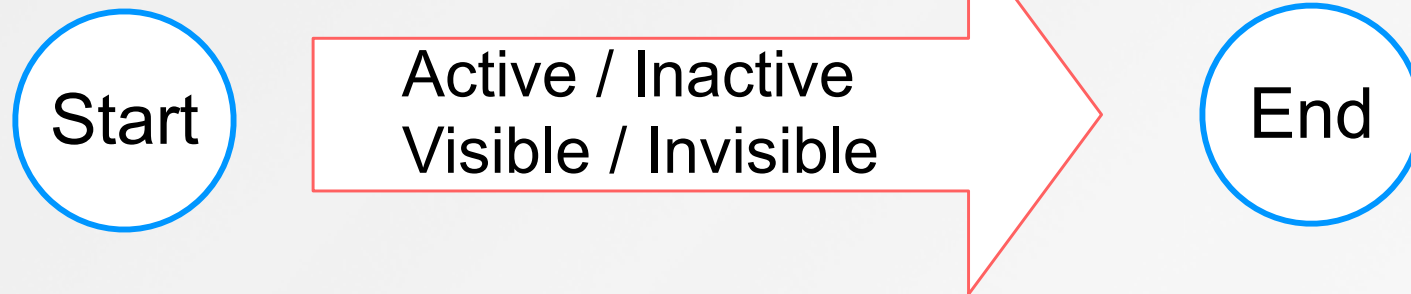
- » Each Android application runs inside its own instance of a Dalvik Virtual Machine (DVM).
- » Unlike a common Windows or Unix process, an Android app does not completely control the completion of its lifecycle
- » Occasionally *hardware resources* may become critically low and the OS could order early termination of any process. The decision considers factors such as:
 - » Number of the application's components currently running.
 - » How long the application's components currently running.
 - » Relative importance of those components to the user.
 - » How much free memory is available in the system.

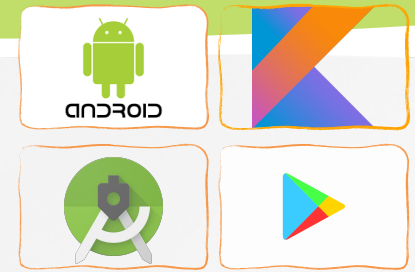


Component's Life Cycle

Life and Death in Android

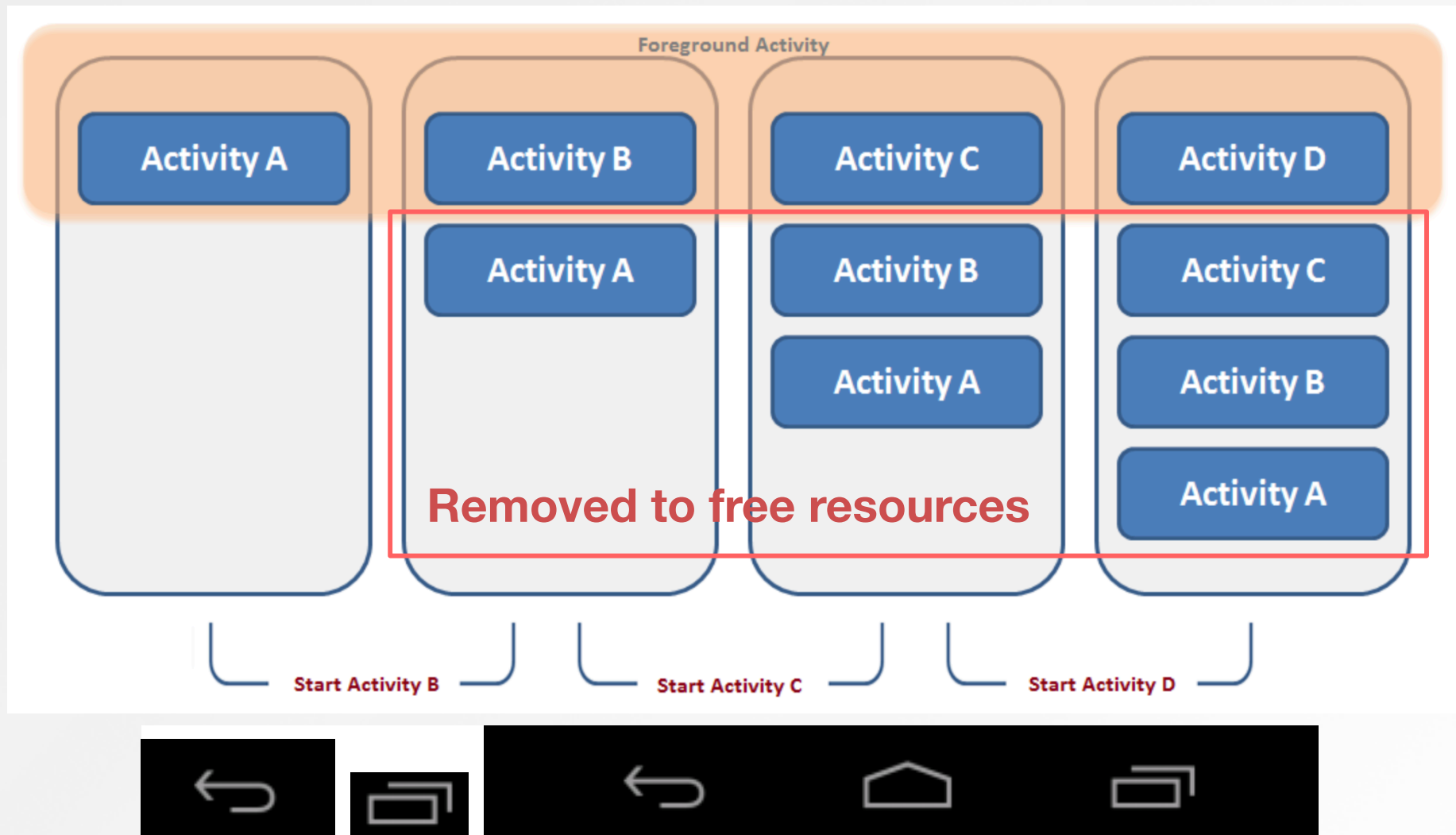
» Main plan

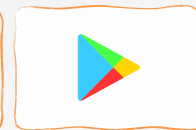




Component's Life Cycle

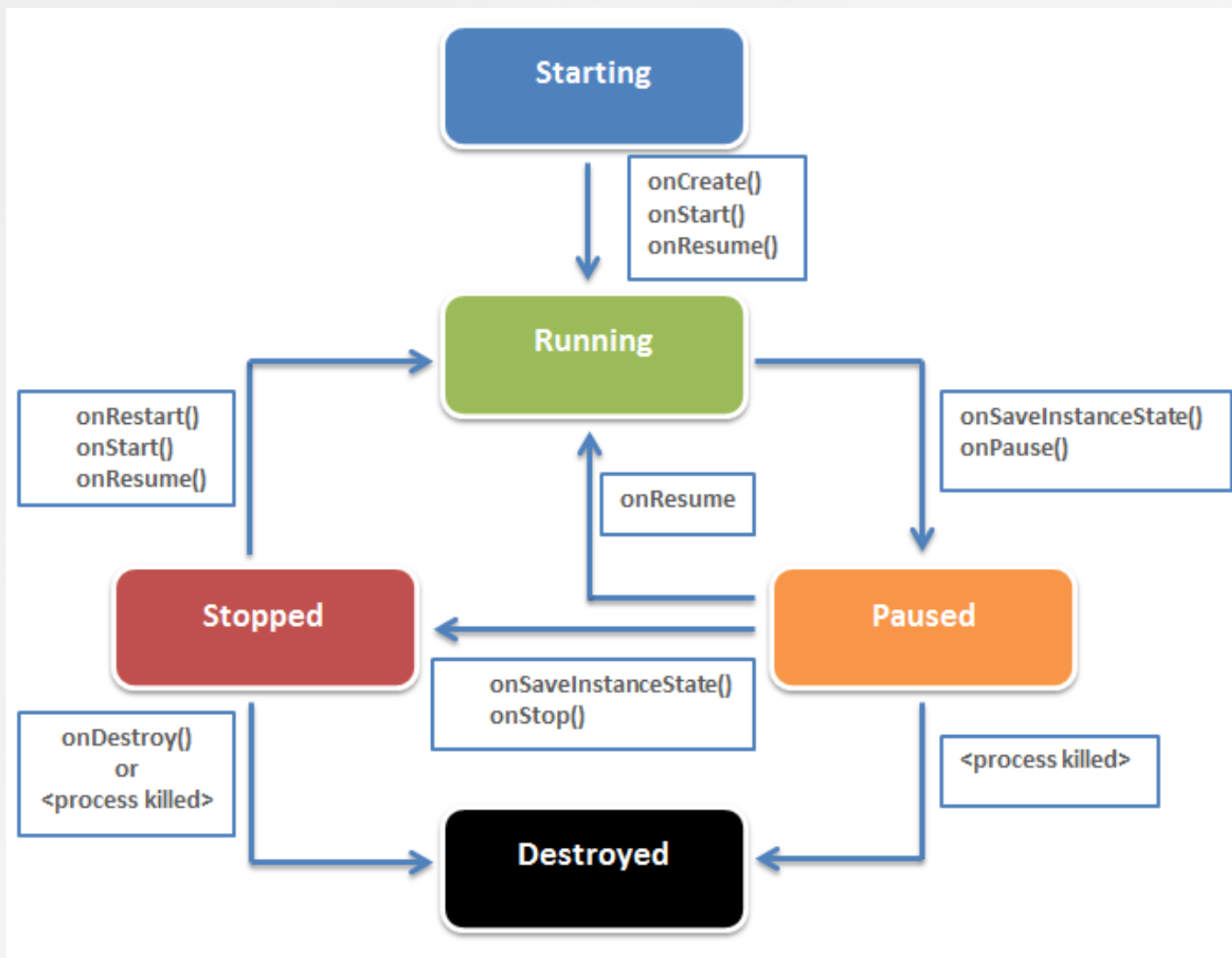
The Activity Stack





Component's Life Cycle

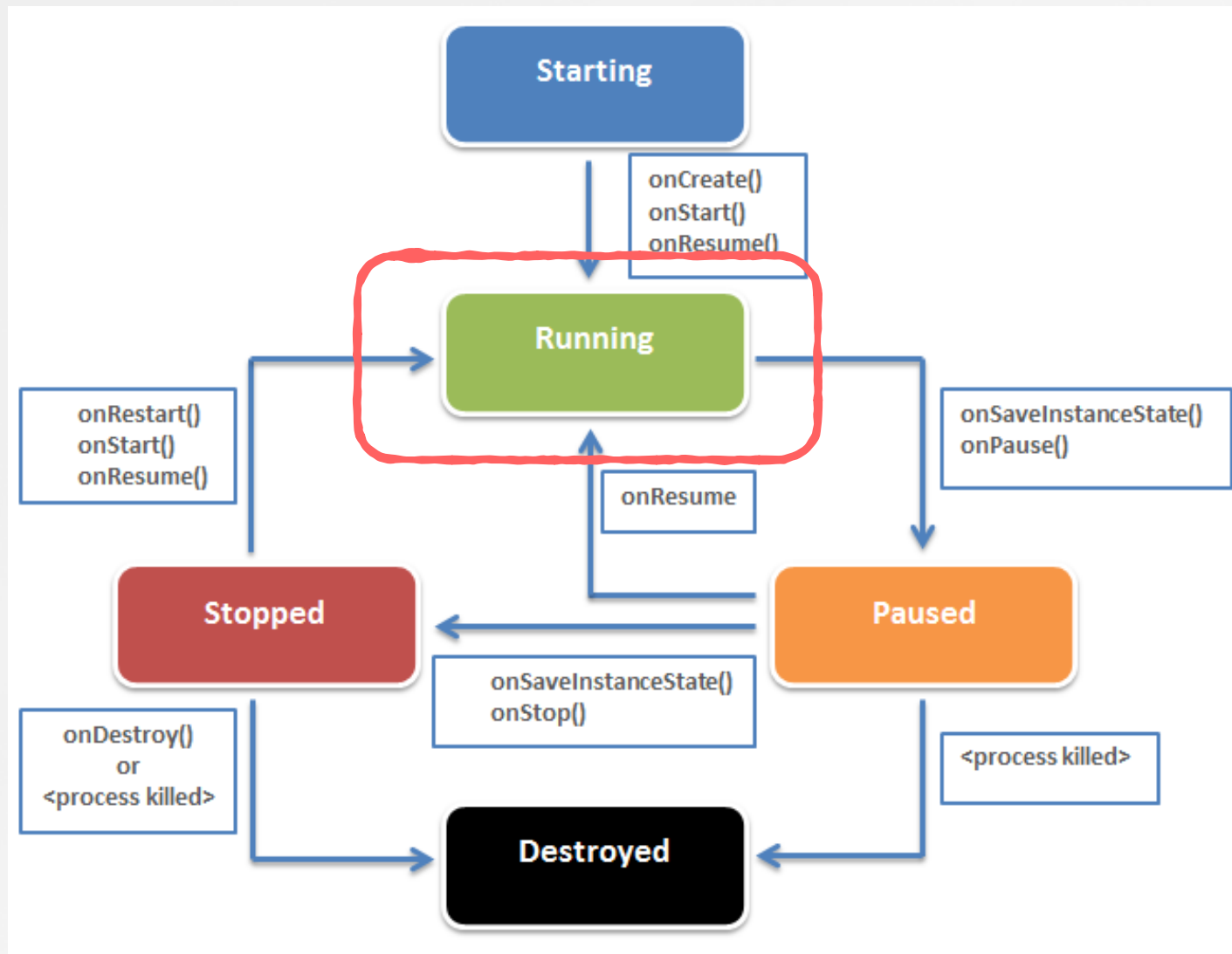
Life Cycle Callbacks





Component's Life Cycle

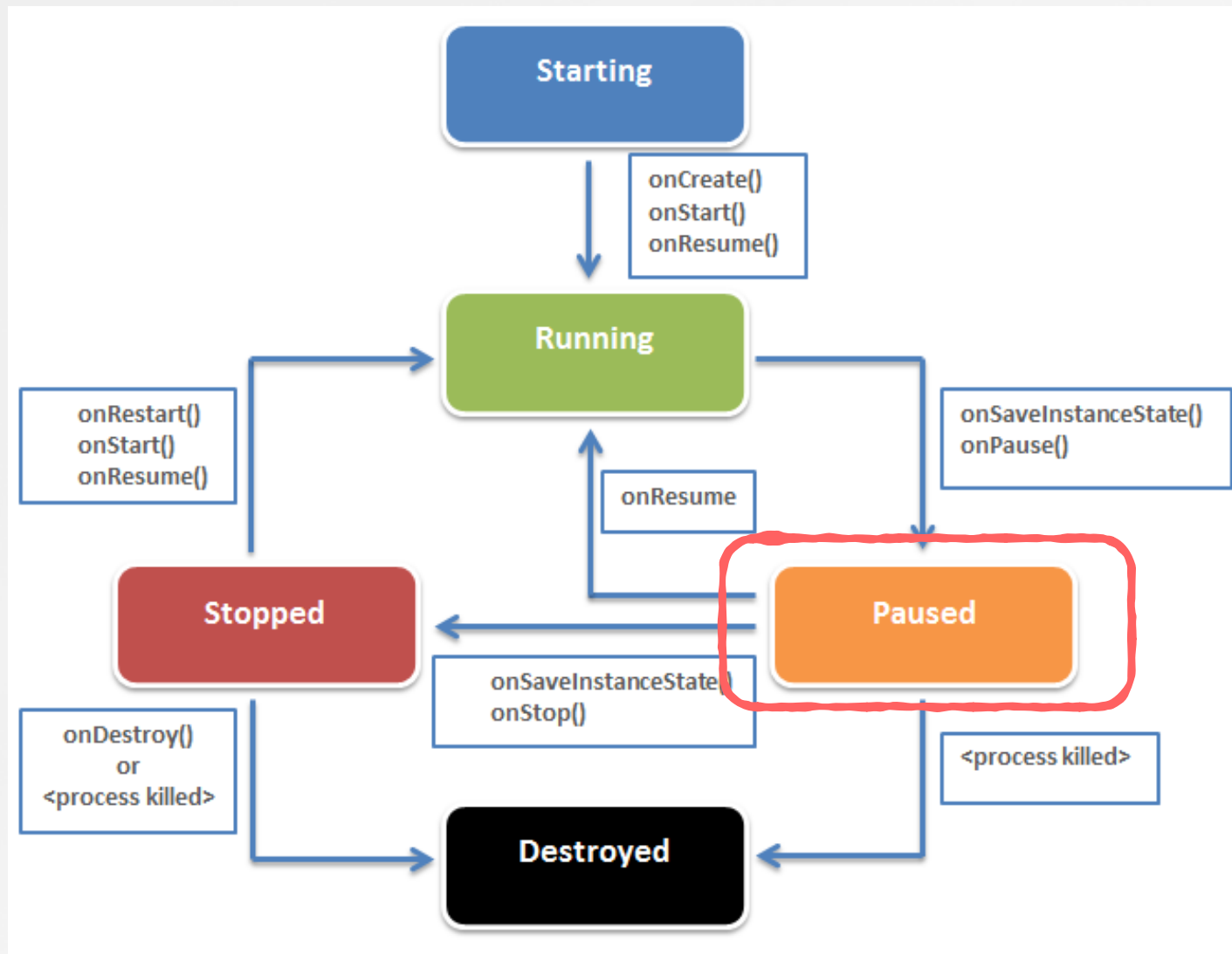
Life Cycle Callbacks





Component's Life Cycle

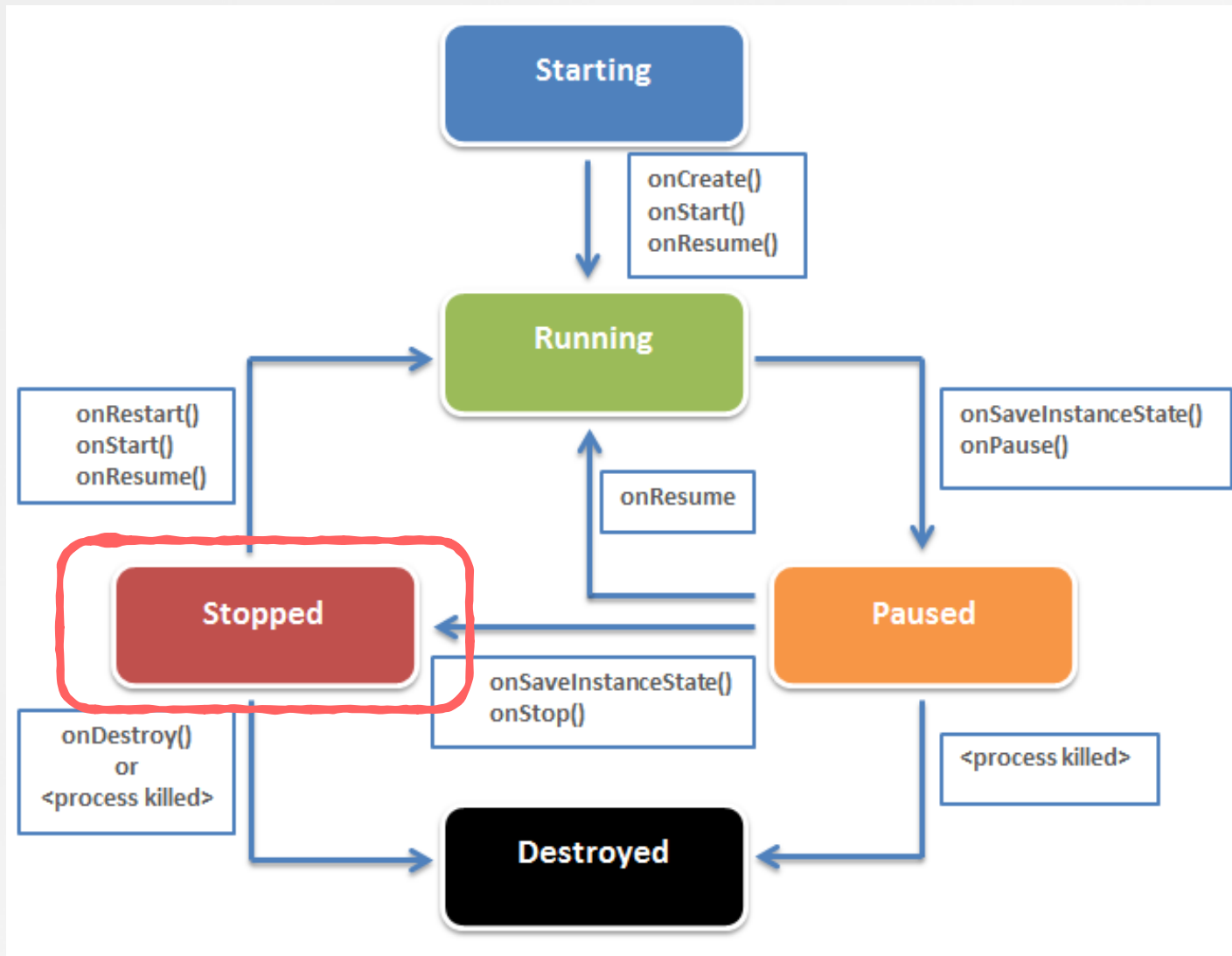
Life Cycle Callbacks

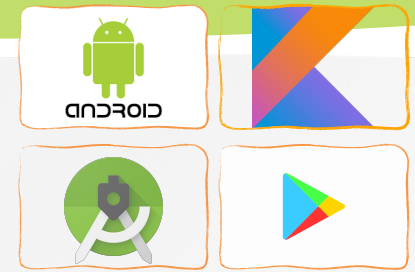




Component's Life Cycle

Life Cycle Callbacks

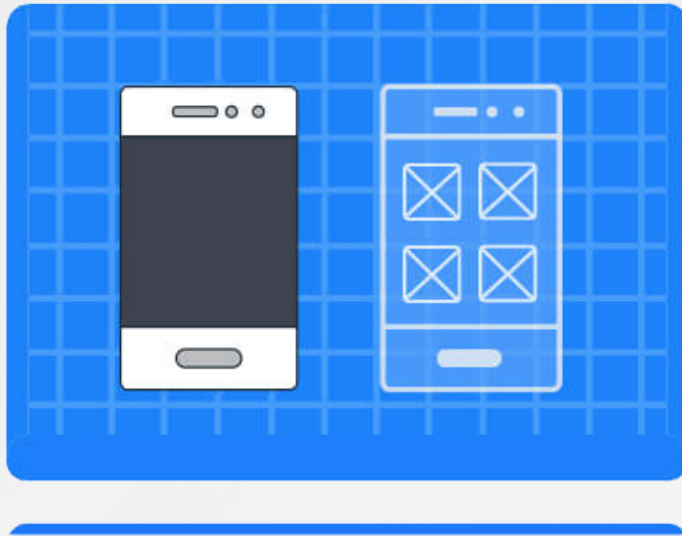
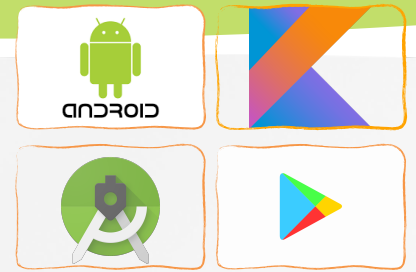


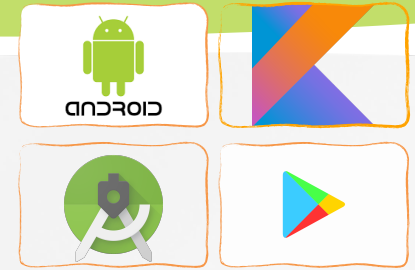


Activity states

Likelihood of being killed	Process state	Activity state
Least	Foreground (having or about to get focus)	Created Started Resumed
More	Background (lost focus)	Paused
Most	Background (not visible)	Stopped
Most	Empty	Destroyed

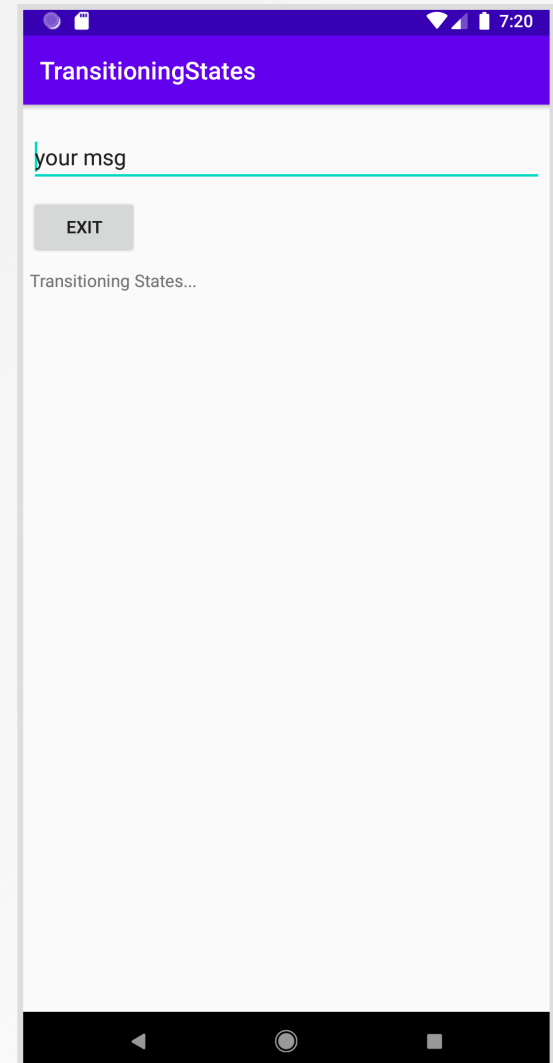
Lab 4.1

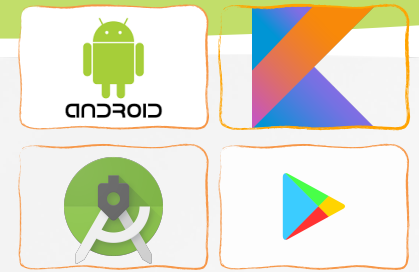




Lab 4.1: Transitioning States

- » Create '*TransitioningStates*' app and do the following scenerios:
 - » The activity_main.xml layout should include an EditText box (msgET), a button (exitBtn), and a TextView (contentTV)
 - » Use the **onCreate** method to connect the button and textbox to the program. Add the following line of code:
Log.i("d", "onCreate")
 - » The **onClick** method has only one command: **finish()**; called to terminate the application.
 - » Add log commands (as the one above) to each of the remaining six main events: onStart, onResume, onPause, onStop, onDestroy, onRestart
 - » Run the application and see the result.

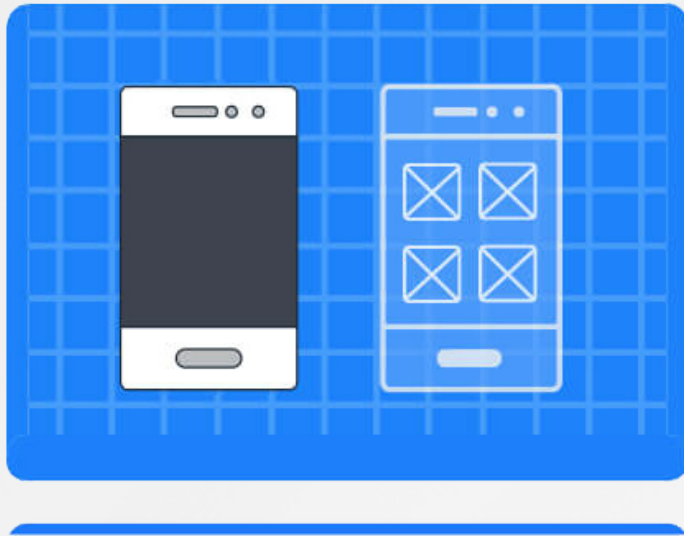
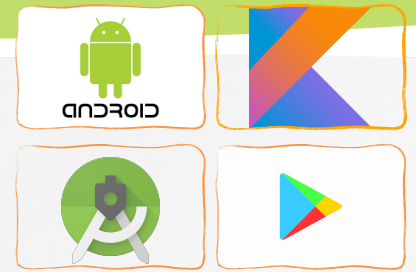


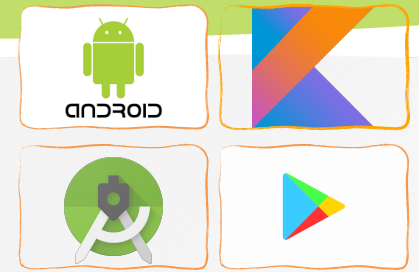


Lab 4.1: Transitioning States

- » Press the EXIT button. Observe the sequence of states displayed.
- » Re-execute the application
- » Press emulator's HOME button. What happens?
- » Click on launch pad, look for the app's icon and return to the app. What sequence of messages is displayed?
- » Make emulator's CALL. Is the app paused or stopped?
- » Click on the BACK button to return to the application.
- » Return the phone call. What happens?
- » Send a text-message to the emulator. What happens?

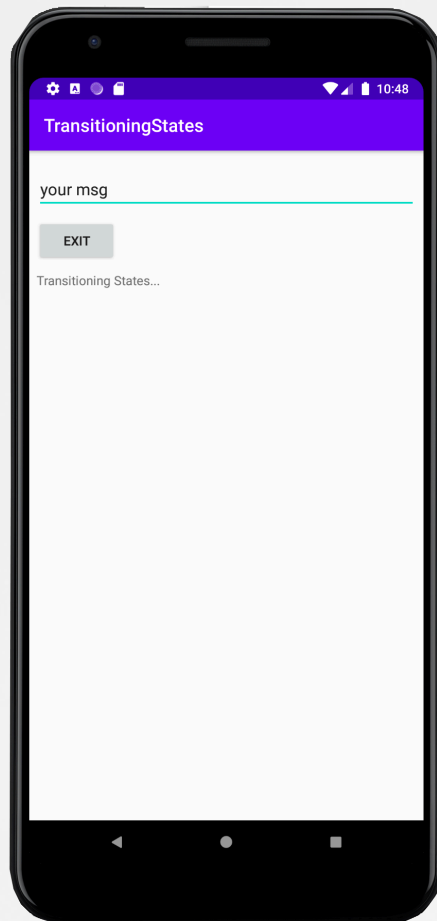
Demo



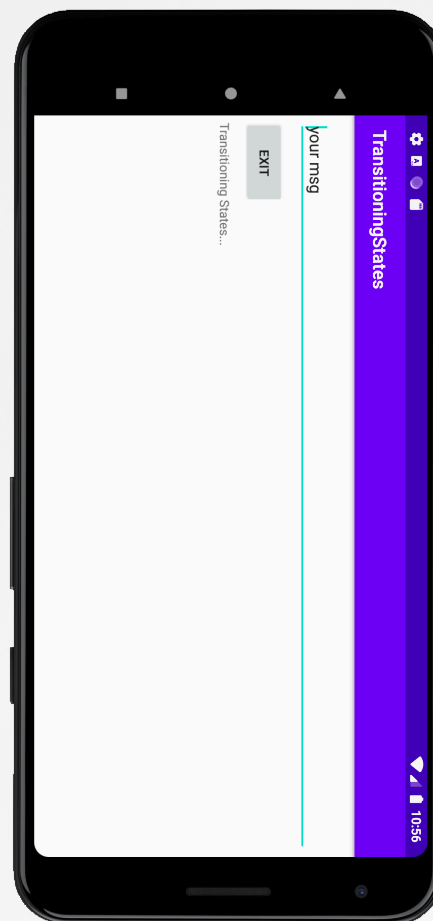


Demo 01: Detecting Device Rotation

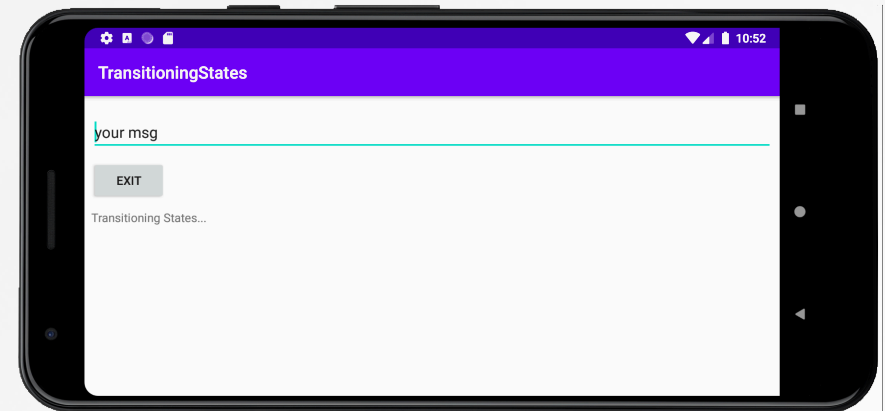
0 - North



2 - South

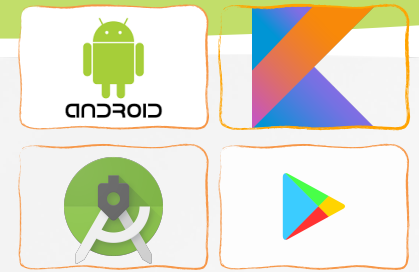


1 - West



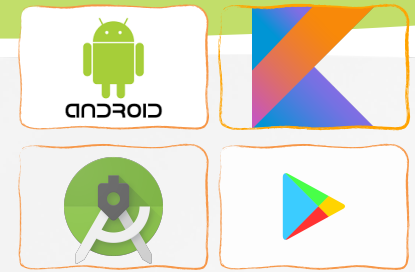
3 - East





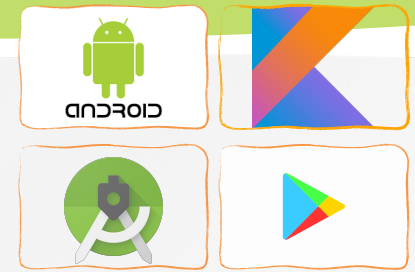
Demo 02: Localization

- » Android apps can run on many devices in many regions.
 - » App should handle text, audio files, numbers, currency, and graphics in ways appropriate to the locales.
- » It is good practice to use the Android resource framework:
 - » Put most or all of the *contents* of your app's user interface into resource files
 - » The *behavior* of the user interface, on the other hand, is driven by your Java-based code



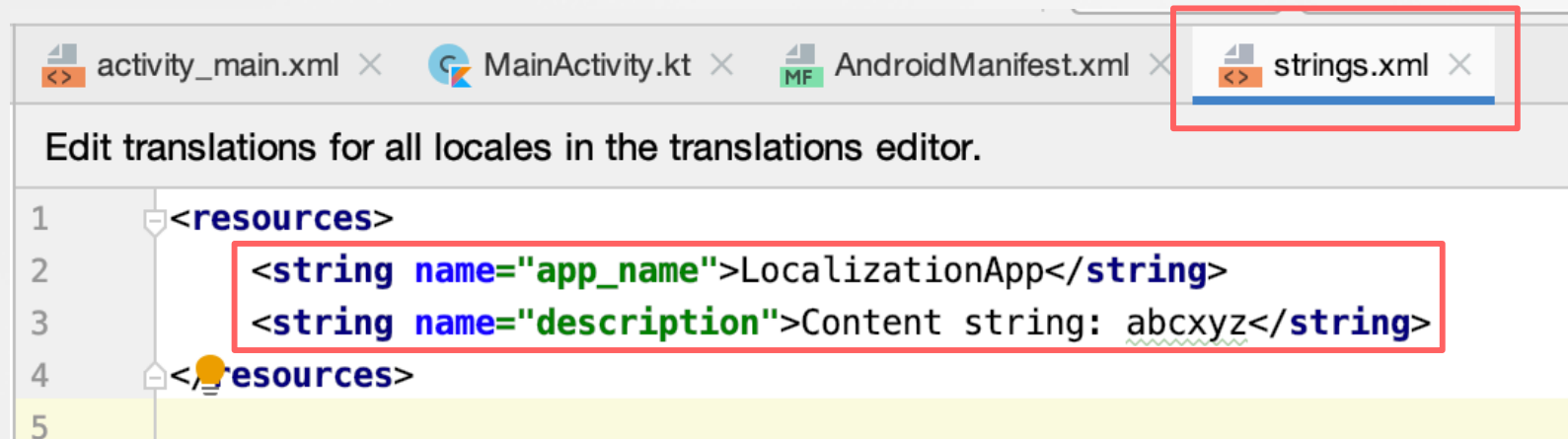
Demo 02: Localization (cont)

- » Android will load the default locale if you have not provided locale for specific text, resources...
 - » The text strings in res/values/strings.xml
 - » If this default file is absent, or if it's missing a string that your app needs, then your app doesn't run and shows an error.
- » The default resource set must also include any default drawables and layouts:
 - » res/drawable/ (required directory holding at least one graphic file, for the app's icon on Google Play)
 - » res/layout/ (required directory holding an XML file that defines the default layout)
 - » res/anim/ (res/anim-*<qualifiers>* folders)
 - » res/xml/ (res/xml-*<qualifiers>* folders)
 - » res/raw/ (res/raw-*<qualifiers>* folders)
 - ...

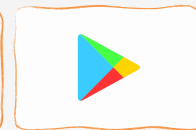


Demo 02: Localization (cont)

» Create default resources:

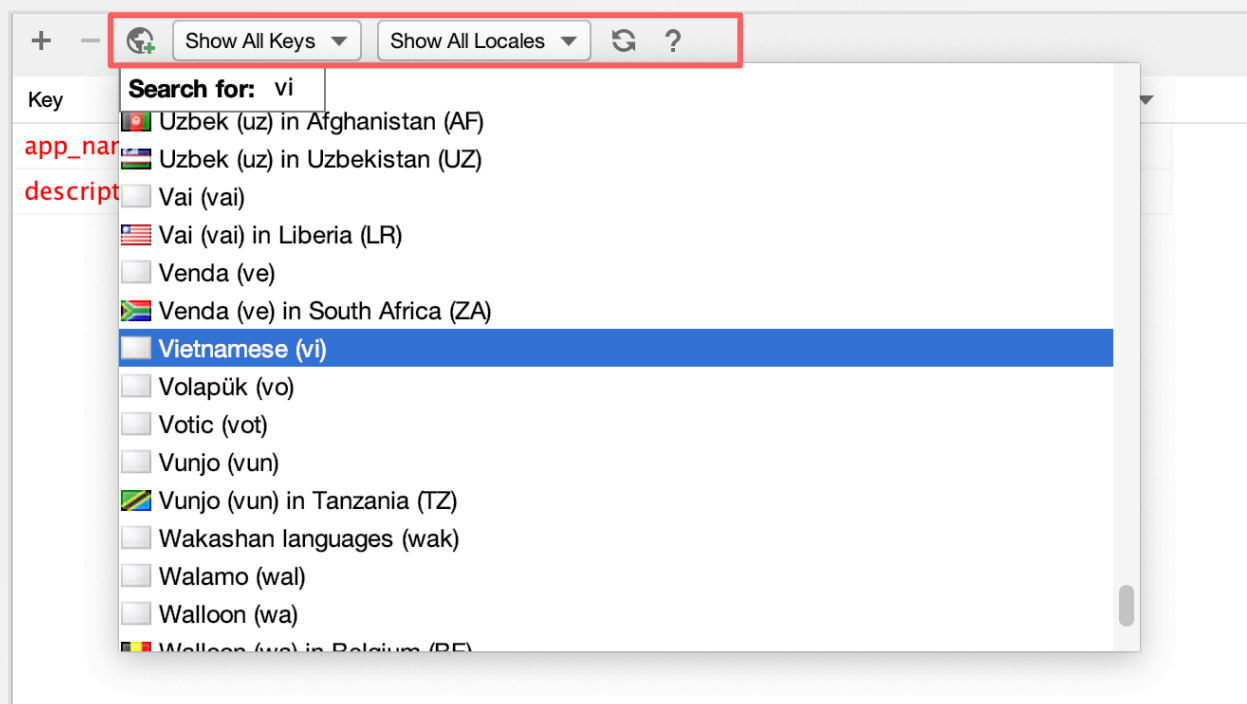


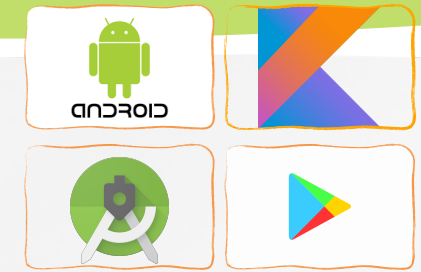
```
activity_main.xml x MainActivity.kt x MF AndroidManifest.xml x strings.xml x
Edit translations for all locales in the translations editor.
1 <resources>
2   <string name="app_name">LocalizationApp</string>
3   <string name="description">Content string: abcxyz</string>
4 </resources>
5
```



Demo 02: Localization (cont)

» Use Translations Editor





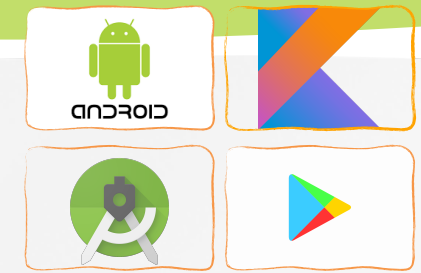
Demo 02: Localization (cont)

» Use Translations Editor

Search for: vi

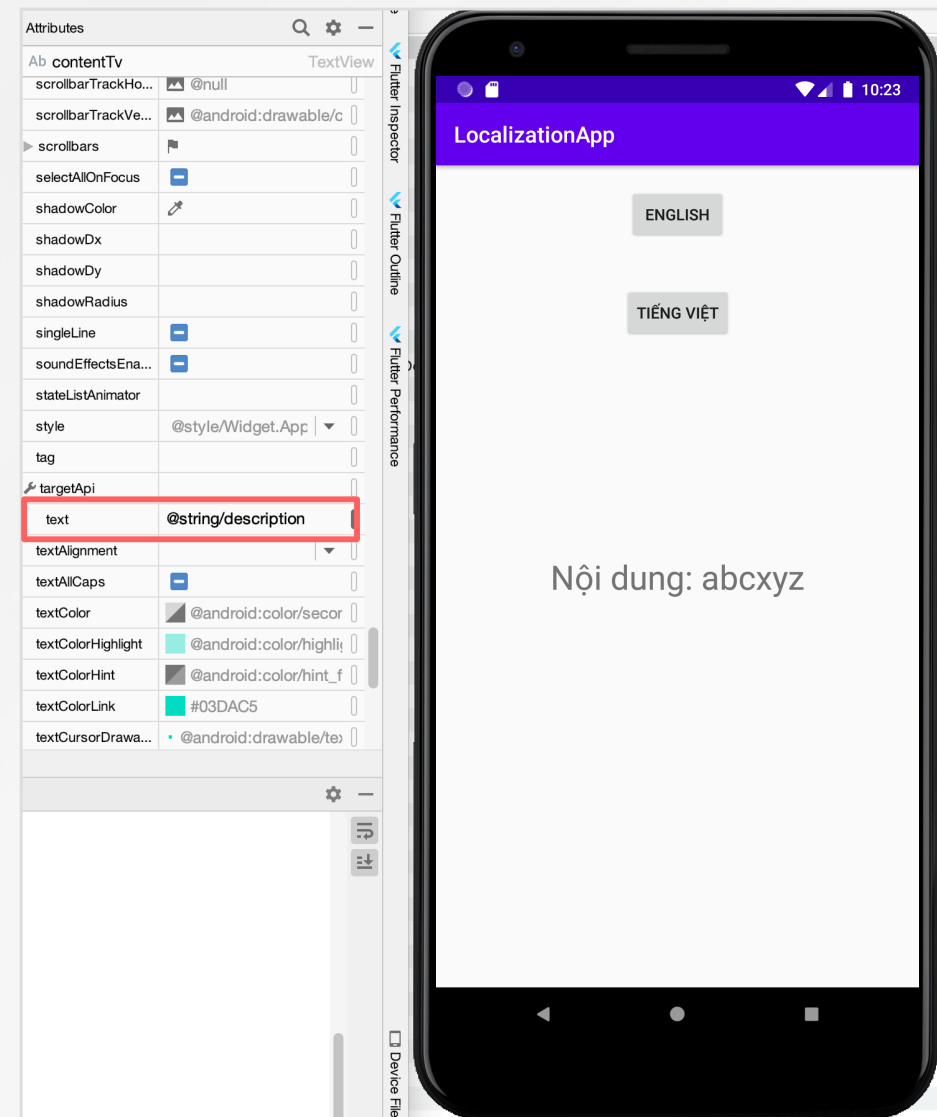
- Uzbek (uz) in Afghanistan (AF)
- Uzbek (uz) in Uzbekistan (UZ)
- Vai (vai)
- Vai (vai) in Liberia (LR)
- Venda (ve)
- Venda (ve) in South Africa (ZA)
- Vietnamese (vi)**
- Volapük (vo)
- Votic (vot)
- Vunjo (vun)
- Vunjo (v)
- Wakash
- Walamo
- Walloon
- Walloon

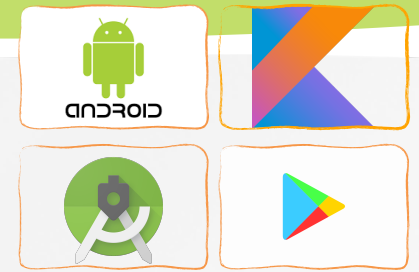
Key	Resource Folder	Untranslatable	Default Value	Vietnamese (vi) in Vietnam (VN)
app_name	app/src/main/res	☐	LocalizationApp	Đa Ngôn Ngữ
description	app/src/main/res	☐	Content string: abcxyz	Nội dung: abcxyz
untranslatable	app/src/main/res	☒	this is untranslatable te	<u>this is untranslatable tex</u>



Demo 02: Localization (cont)

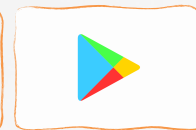
- » Design UI using
 - » ‘@string/description’ for **text** attribute
 - » changeLangBtn
 - » contentTv
- » Run app
- » Chang language in Settings and check your screen





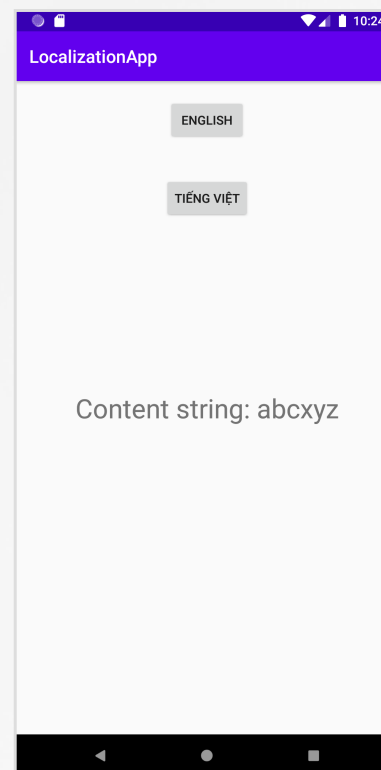
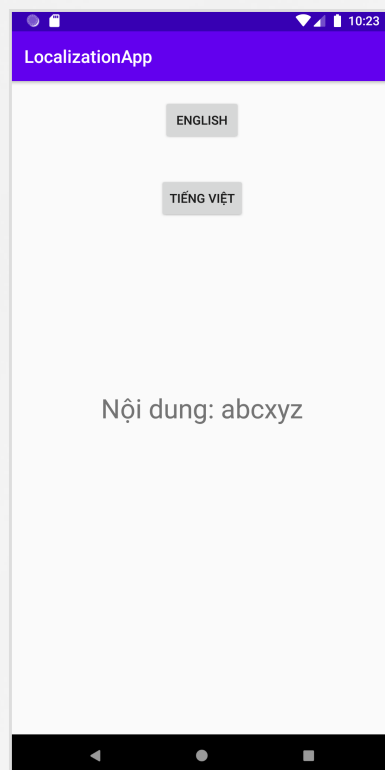
Demo 02: Localization (cont)

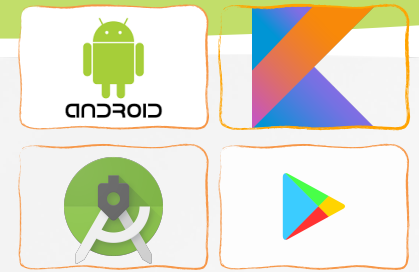
- » Design UI using '@string/description' for **text** attribute
 - » changeLangBtn
 - » contentTv
- » Run app
- » Chang language in Settings and check your screen



Demo 02: Localization (cont)

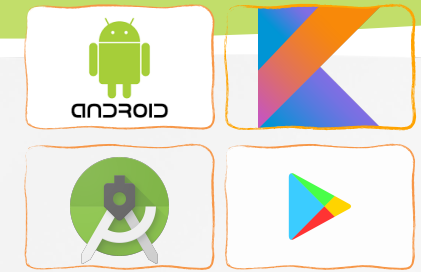
- » 1st improvement: change language at runtime
 - » 'LocalizationApp01-OnlyChangeLangAndText.zip'





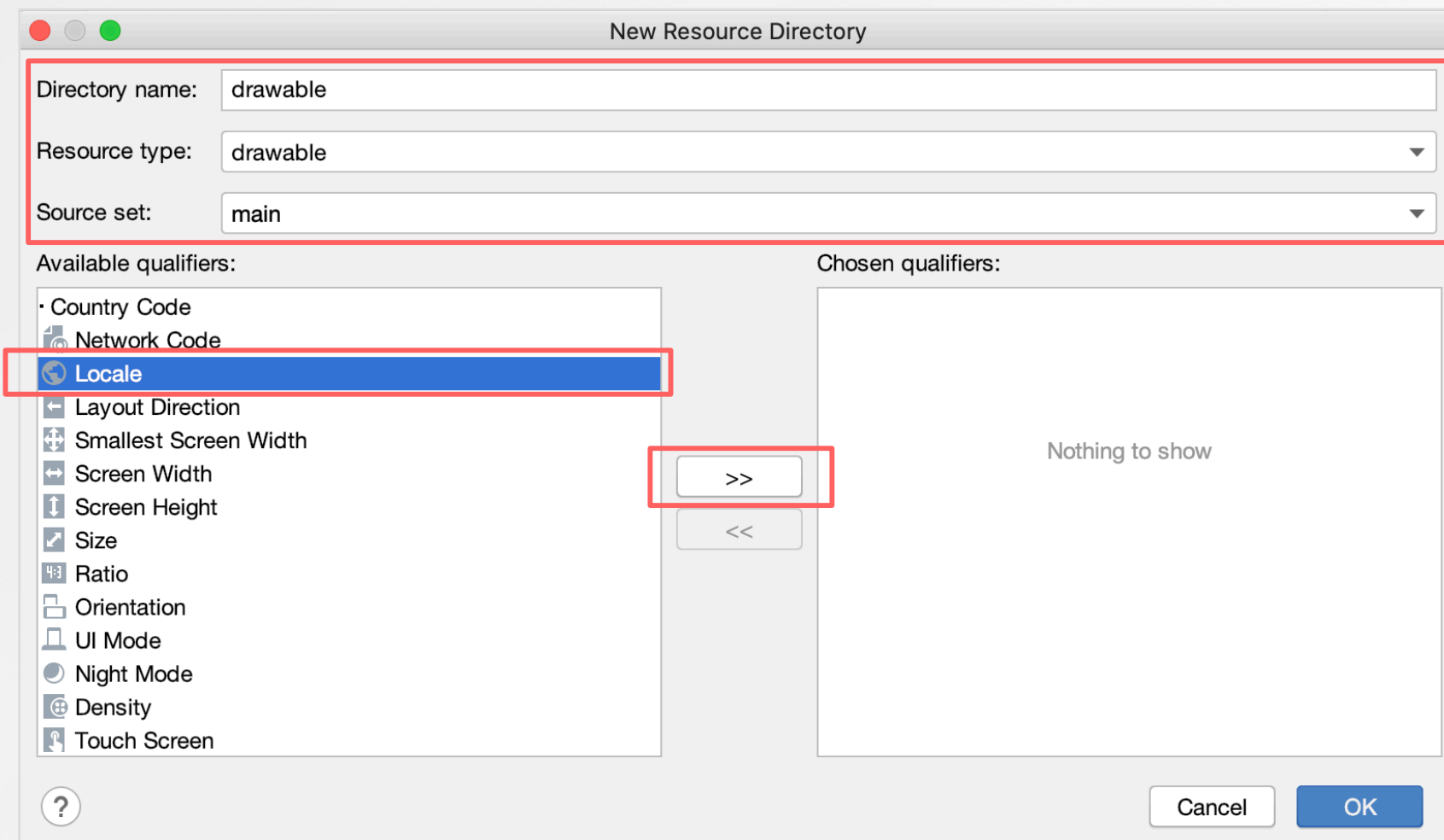
Demo 02: Localization (cont)

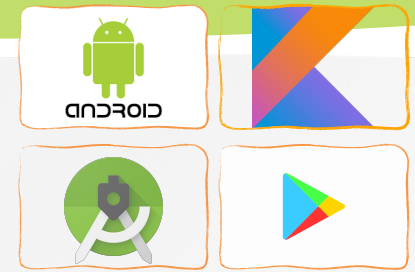
- » 2nd improvement: change drawable (images)
 - » Right click on the **res** package and select **New ▶ Android resource directory**.
 - » You need to create a new drawable folder for the **vi** locale:
 - » **drawable** as the resource type
 - » **locale** as the qualifier
 - » **vi** as the language
 - » **Any region** as the region



Demo 02: Localization (cont)

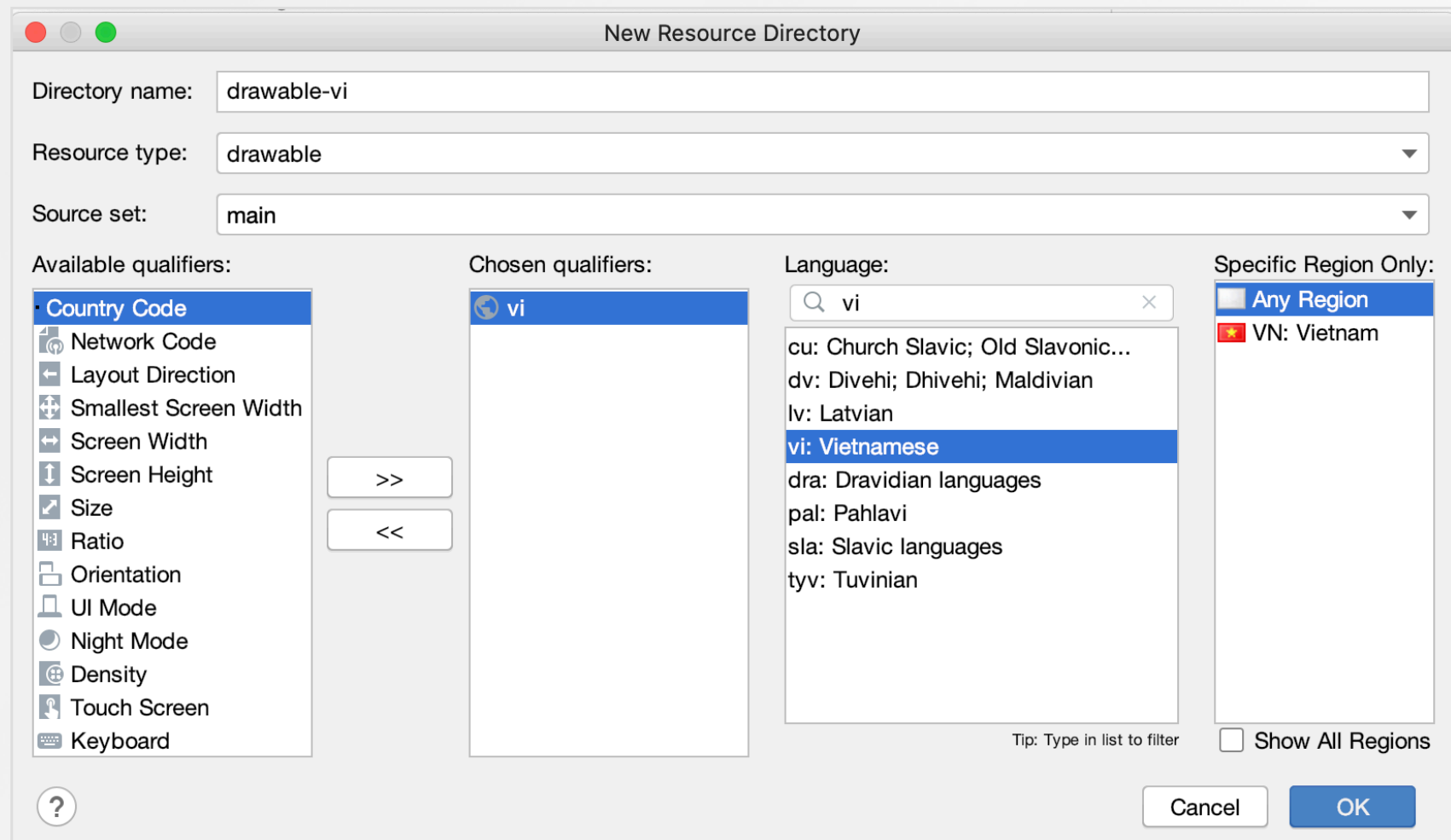
» 2nd improvement: change drawable (images)

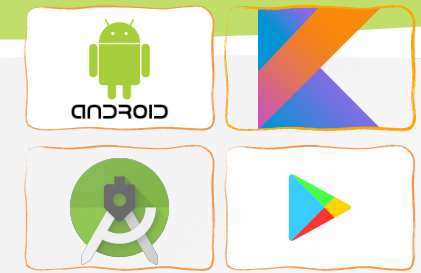




Demo 02: Localization (cont)

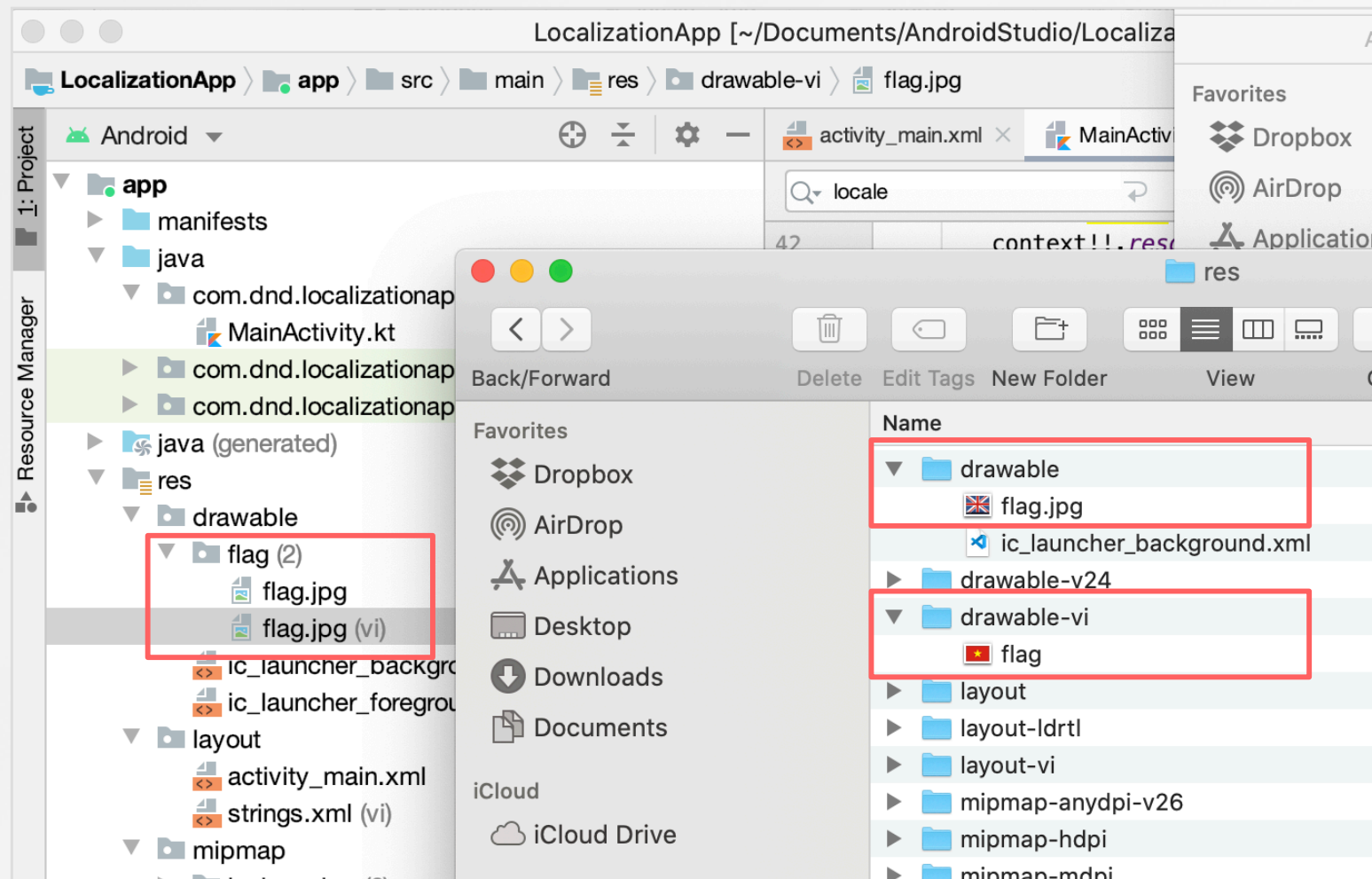
» 2nd improvement: change drawable (images)

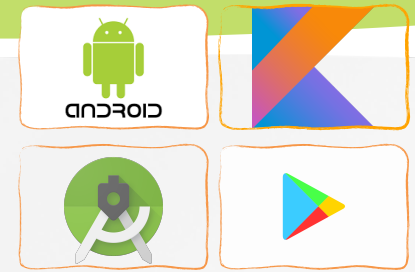




Demo 02: Localization (cont)

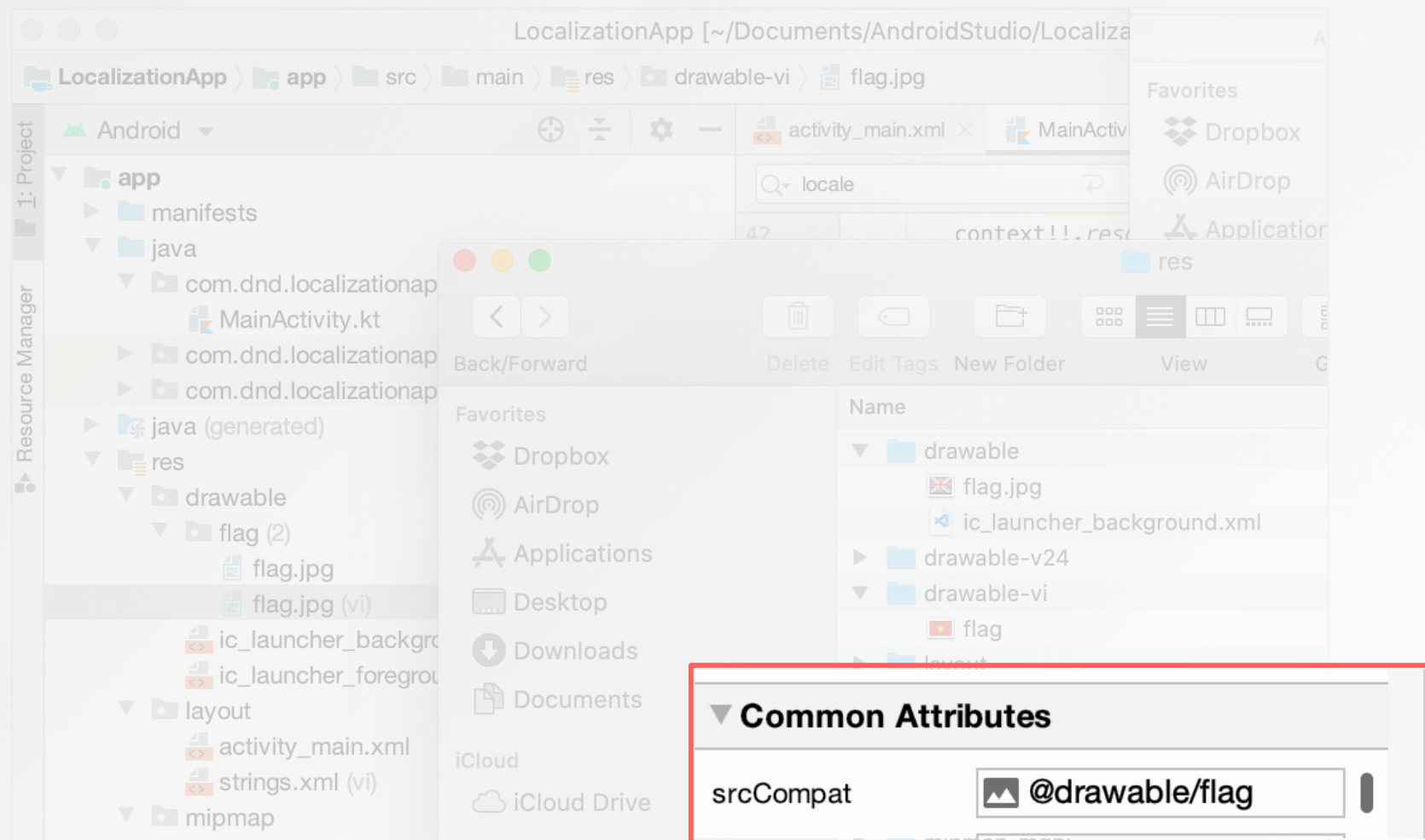
» 2nd improvement: change drawable (images)

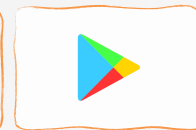




Demo 02: Localization (cont)

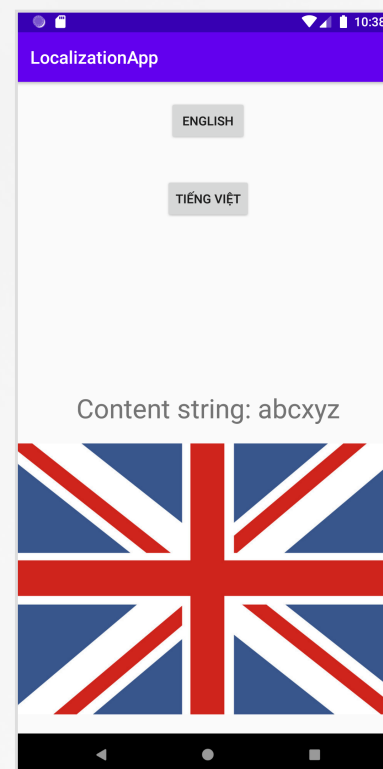
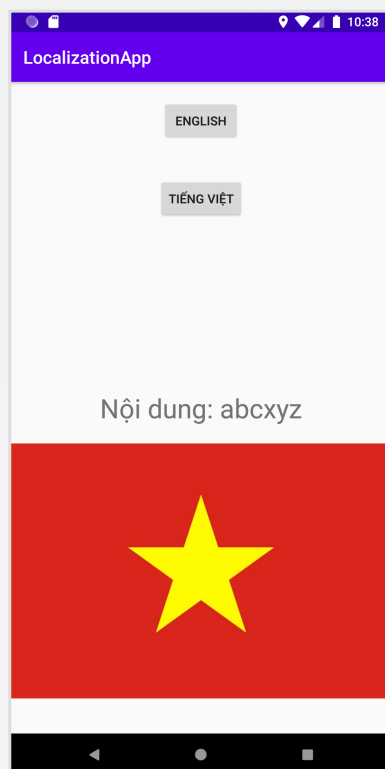
» 2nd improvement: change drawable (images)



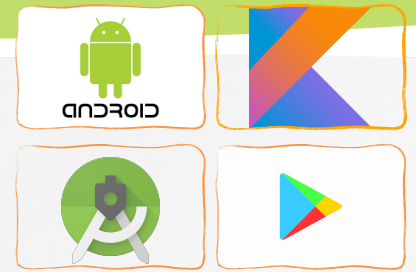


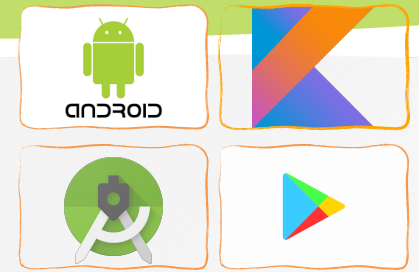
Demo 02: Localization (cont)

- » 2nd improvement: change drawable (images)
 - » 'LocalizationApp02-BothTextAndDrawable.zip'



Q&A





Reference

- » <https://kotlinlang.org/docs/reference/>
- » Peter Späth, Learn Kotlin for Android Development, 2019
- » Ted Hagos, Learn Android Studio 3 with Kotlin, 2018
- » Peter Späth, Pro Android with Kotlin, 2019
- » Milos Vasic, Mastering Android Development with Kotlin, 2017
- » <https://developer.android.com/guide/>
- » <https://www.raywenderlich.com/4878024-android-localization-getting-started>